

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    roc_auc_score
)

```

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc.data"
```

```

columns = [
    "ID",
    "Diagnosis",
    "radius_mean",
    "texture_mean",
    "perimeter_mean",
    "area_mean",
    "smoothness_mean",
    "compactness_mean",
    "concavity_mean",
    "concave_points_mean",
    "symmetry_mean",
    "fractal_dimension_mean",

    "radius_se",
    "texture_se",
    "perimeter_se",
    "area_se",
    "smoothness_se",
    "compactness_se",
    "concavity_se",
    "concave_points_se",
    "symmetry_se",
    "fractal_dimension_se",

    "radius_worst",
    "texture_worst",
    "perimeter_worst",
    "area_worst",
    "smoothness_worst",
    "compactness_worst",
    "concavity_worst",
    "concave_points_worst",
    "symmetry_worst",
    "fractal_dimension_worst"
]

```

```
df = pd.read_csv(url, header=None, names=columns)
```

```

print("Dataset loaded successfully!")
print("Shape:", df.shape)

```

```
df.head()
```

Dataset loaded successfully!

Shape: (569, 32)

	ID	Diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	cor
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	

5 rows × 32 columns

df.info()

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 569 entries, 0 to 568

Data columns (total 32 columns):

#	Column	Non-Null Count	Dtype
0	ID	569 non-null	int64
1	Diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave_points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave_points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64
29	concave_points_worst	569 non-null	float64
30	symmetry_worst	569 non-null	float64
31	fractal_dimension_worst	569 non-null	float64

dtypes: float64(30), int64(1), object(1)

memory usage: 142.4+ KB

```
print("First 5 rows:")
display(df.head())
```

```
print("\nLast 5 rows:")
display(df.tail())
```

First 5 rows:

	ID	Diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	cor
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	

5 rows × 32 columns

Last 5 rows:

	ID	Diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	cor
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	

5 rows × 32 columns

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Number of rows: 569
Number of columns: 32
```

```
print(df["Diagnosis"].unique())
```

```
['M' 'B']
```

```
# summary
```

```
X = df.drop(columns=["ID", "Diagnosis"])
y = df["Diagnosis"]
```

```
print("Number of features:", X.shape[1])
```

```
Number of features: 30
```

```
# mean, median, standard deviation
```

```
statistics = pd.DataFrame({
    "Mean": X.mean(),
    "Median": X.median(),
    "Standard Deviation": X.std(),
    "Minimum": X.min(),
    "Maximum": X.max()
})
```

```
statistics
```

	Mean	Median	Standard Deviation	Minimum	Maximum	
radius_mean	14.127292	13.370000	3.524049	6.981000	28.11000	
texture_mean	19.289649	18.840000	4.301036	9.710000	39.28000	
perimeter_mean	91.969033	86.240000	24.298981	43.790000	188.50000	
area_mean	654.889104	551.100000	351.914129	143.500000	2501.00000	
smoothness_mean	0.096360	0.095870	0.014064	0.052630	0.16340	
compactness_mean	0.104341	0.092630	0.052813	0.019380	0.34540	
concavity_mean	0.088799	0.061540	0.079720	0.000000	0.42680	
concave_points_mean	0.048919	0.033500	0.038803	0.000000	0.20120	
symmetry_mean	0.181162	0.179200	0.027414	0.106000	0.30400	
fractal_dimension_mean	0.062798	0.061540	0.007060	0.049960	0.09744	
radius_se	0.405172	0.324200	0.277313	0.111500	2.87300	
texture_se	1.216853	1.108000	0.551648	0.360200	4.88500	
perimeter_se	2.866059	2.287000	2.021855	0.757000	21.98000	
area_se	40.337079	24.530000	45.491006	6.802000	542.20000	
smoothness_se	0.007041	0.006380	0.003003	0.001713	0.03113	
compactness_se	0.025478	0.020450	0.017908	0.002252	0.13540	
concavity_se	0.031894	0.025890	0.030186	0.000000	0.39600	
concave_points_se	0.011796	0.010930	0.006170	0.000000	0.05279	
symmetry_se	0.020542	0.018730	0.008266	0.007882	0.07895	
fractal_dimension_se	0.003795	0.003187	0.002646	0.000895	0.02984	
radius_worst	16.269190	14.970000	4.833242	7.930000	36.04000	
texture_worst	25.677223	25.410000	6.146258	12.020000	49.54000	
perimeter_worst	107.261213	97.660000	33.602542	50.410000	251.20000	
area_worst	880.583128	686.500000	569.356993	185.200000	4254.00000	
smoothness_worst	0.132369	0.131300	0.022832	0.071170	0.22260	
compactness_worst	0.254265	0.211900	0.157336	0.027290	1.05800	
concavity_worst	0.272188	0.226700	0.208624	0.000000	1.25200	
concave_points_worst	0.114606	0.099930	0.065732	0.000000	0.29100	
symmetry_worst	0.290076	0.282200	0.061867	0.156500	0.66380	
fractal_dimension_worst	0.083946	0.080040	0.018061	0.055040	0.20750	

Next steps: [Generate code with statistics](#) [New interactive sheet](#)

```
# checking missing values
```

```
missing_values = df.isnull().sum()
```

```
print("Missing values in each column:")
```

```
print(missing_values)
```

```
Missing values in each column:
```

```
ID          0
Diagnosis   0
radius_mean 0
texture_mean 0
perimeter_mean 0
area_mean   0
smoothness_mean 0
compactness_mean 0
concavity_mean 0
concave_points_mean 0
symmetry_mean 0
fractal_dimension_mean 0
radius_se   0
texture_se   0
perimeter_se 0
area_se     0
smoothness_se 0
compactness_se 0
concavity_se 0
concave_points_se 0
```

```
symmetry_se      0
fractal_dimension_se  0
radius_worst     0
texture_worst    0
perimeter_worst  0
area_worst       0
smoothness_worst 0
compactness_worst 0
concavity_worst  0
concave_points_worst 0
symmetry_worst   0
fractal_dimension_worst 0
dtype: int64
```

```
# checking duplicates
```

```
print("Number of duplicate rows:", df.duplicated().sum())
```

```
if df.duplicated().sum() > 0:
    df = df.drop_duplicates()
    print("Duplicates removed.")
else:
    print("No duplicate rows found.")
```

```
Number of duplicate rows: 0
No duplicate rows found.
```

```
# checking outliers
```

```
outlier_counts = {}
```

```
for column in X.columns:
    Q1 = X[column].quantile(0.25)
    Q3 = X[column].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
outliers = X[
    (X[column] < lower_bound) |
    (X[column] > upper_bound)
]
```

```
outlier_counts[column] = len(outliers)
```

```
outlier_df = pd.DataFrame.from_dict(
    outlier_counts,
    orient="index",
    columns=["Number of Outliers"]
)
```

```
outlier_df
```

	Number of Outliers	
radius_mean	14	 
texture_mean	7	
perimeter_mean	13	
area_mean	25	
smoothness_mean	6	
compactness_mean	16	
concavity_mean	18	
concave_points_mean	10	
symmetry_mean	15	
fractal_dimension_mean	15	
radius_se	38	
texture_se	20	
perimeter_se	38	
area_se	65	
smoothness_se	30	
compactness_se	28	
concavity_se	22	
concave_points_se	19	
symmetry_se	27	
fractal_dimension_se	28	
radius_worst	17	
texture_worst	5	
perimeter_worst	15	
area_worst	35	
smoothness_worst	7	
compactness_worst	16	
concavity_worst	12	
concave_points_worst	0	
symmetry_worst	23	
fractal_dimension_worst	24	

Next steps: [Generate code with outlier_df](#) [New interactive sheet](#)

```
# box plot for outliers

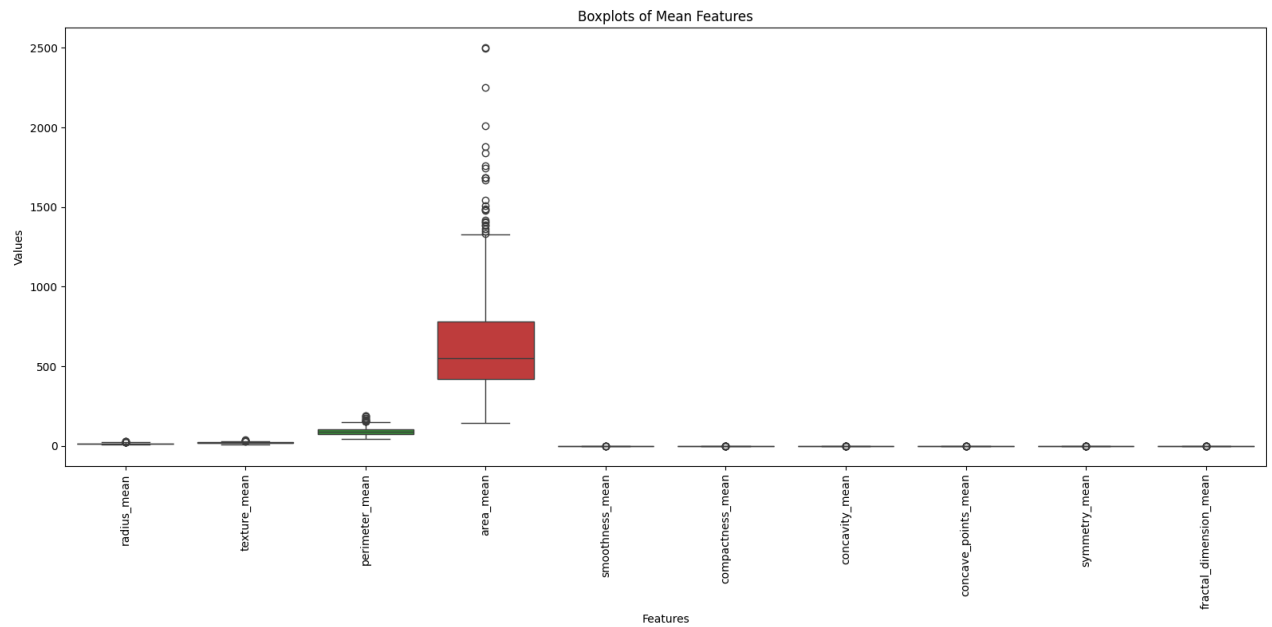
mean_features = [
    col for col in X.columns
    if col.endswith("_mean")
]

plt.figure(figsize=(16, 8))

sns.boxplot(data=X[mean_features])

plt.xticks(rotation=90)
plt.title("Boxplots of Mean Features")
plt.xlabel("Features")
plt.ylabel("Values")

plt.tight_layout()
plt.show()
```



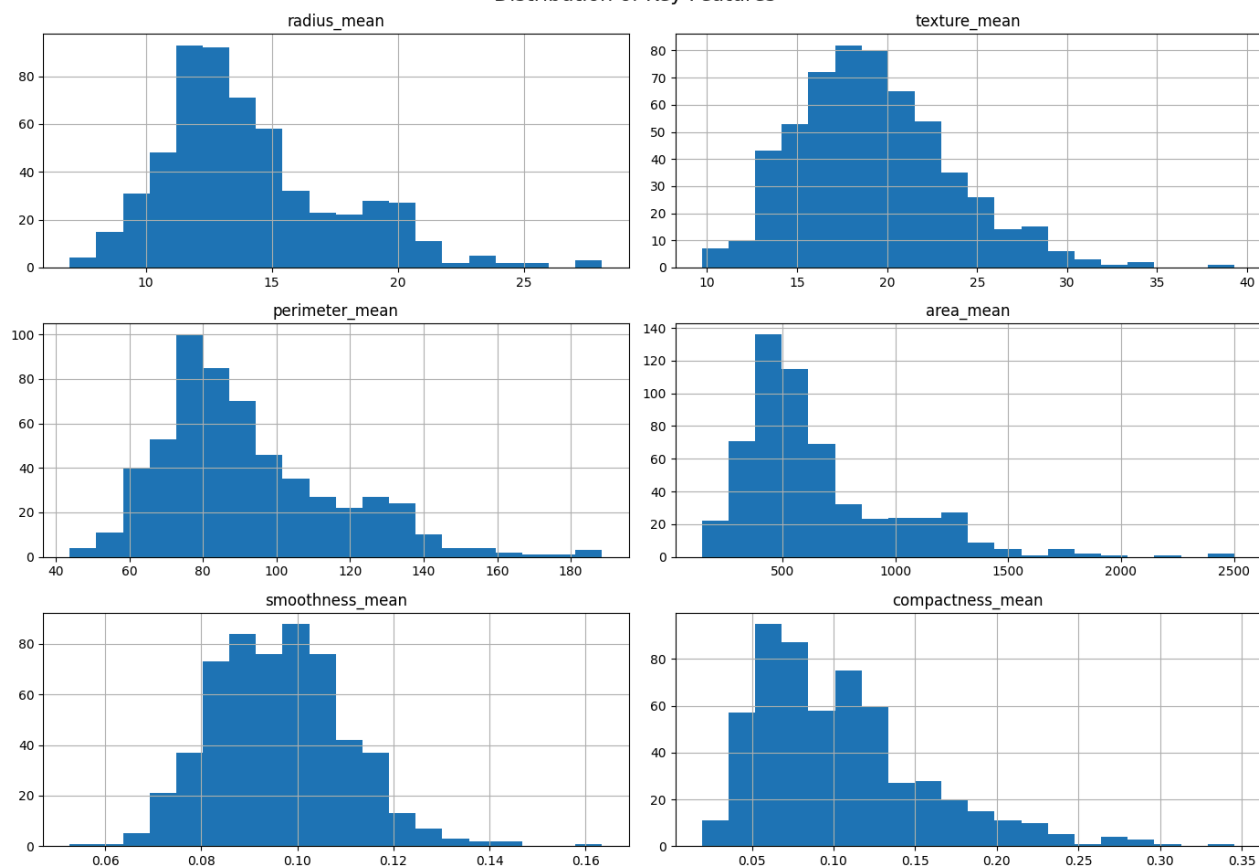
```
# histogram of imp features

key_features = [
    "radius_mean",
    "texture_mean",
    "perimeter_mean",
    "area_mean",
    "smoothness_mean",
    "compactness_mean"
]

X[key_features].hist(
    figsize=(14, 10),
    bins=20
)

plt.suptitle("Distribution of Key Features", fontsize=16)
plt.tight_layout()
plt.show()
```

Distribution of Key Features



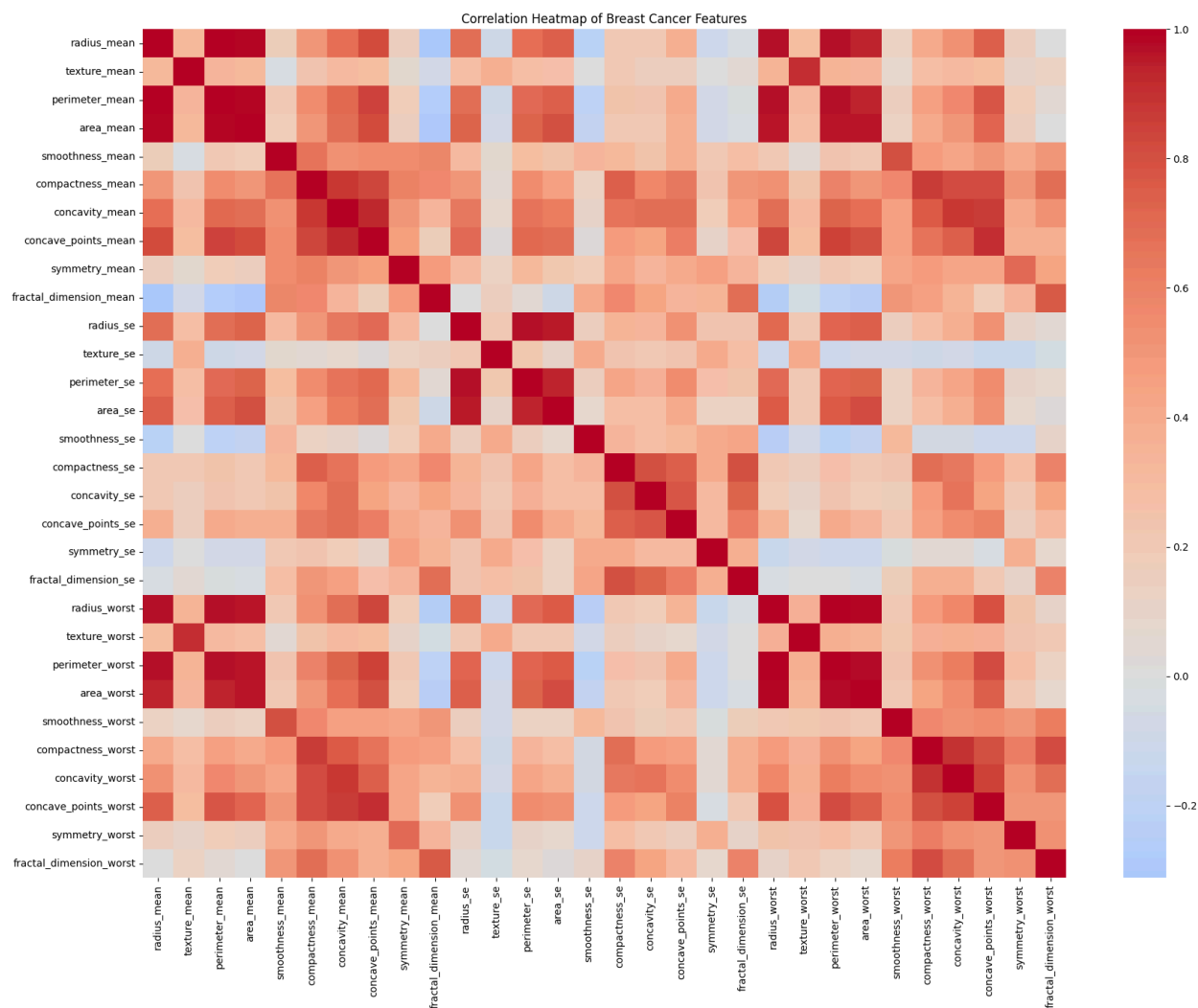
```
# correlation heatmap

plt.figure(figsize=(18, 14))

correlation_matrix = X.corr()

sns.heatmap(
    correlation_matrix,
    cmap="coolwarm",
    center=0
)

plt.title("Correlation Heatmap of Breast Cancer Features")
plt.tight_layout()
plt.show()
```



```
# class distribution
```

```
print(df["Diagnosis"].value_counts())
```

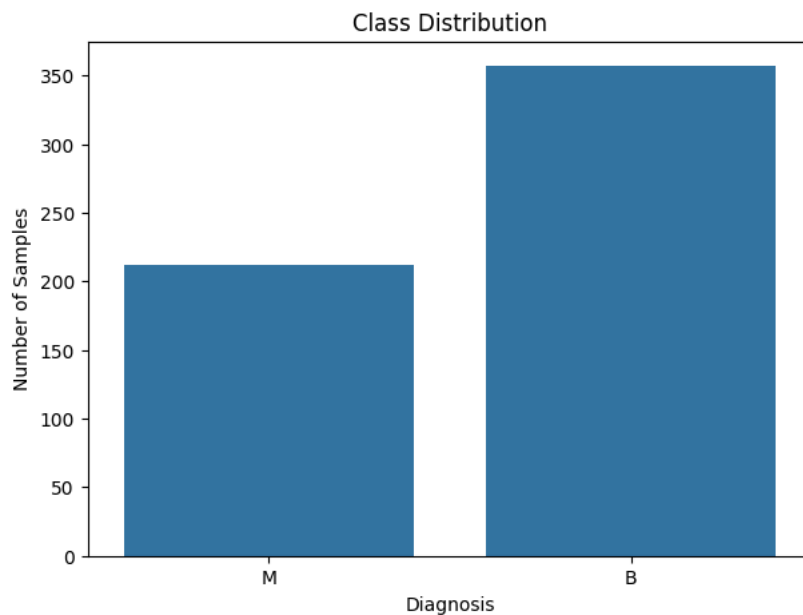
```
Diagnosis
B      357
M      212
Name: count, dtype: int64
```

```
plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="Diagnosis"
)

plt.title("Class Distribution")
plt.xlabel("Diagnosis")
plt.ylabel("Number of Samples")

plt.show()
```



```
class_percentages = df["Diagnosis"].value_counts(normalize=True) * 100
```

```
print(class_percentages)
```

```
Diagnosis
B    62.741652
M    37.258348
Name: proportion, dtype: float64
```

```
# data preprocessing
```

```
df["Outcome"] = df["Diagnosis"].map({
    "B": 0,
    "M": 1
})

print(df[["Diagnosis", "Outcome"]].head(10))
```

	Diagnosis	Outcome
0	M	1
1	M	1
2	M	1
3	M	1
4	M	1
5	M	1
6	M	1
7	M	1
8	M	1
9	M	1

```
print(df["Outcome"].value_counts())
```

```
Outcome
0    357
1    212
Name: count, dtype: int64
```

```
X = df.drop(
    columns=["ID", "Diagnosis", "Outcome"]
)
```

```
y = df["Outcome"]
```

```
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (569, 30)
y shape: (569,)
```

```
# train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
```

```

        random_state=42,
        stratify=y
    )

    print("Training samples:", X_train.shape[0])
    print("Testing samples:", X_test.shape[0])

```

```

Training samples: 455
Testing samples: 114

```

```

print("Training class distribution:")
print(y_train.value_counts(normalize=True))

print("\nTesting class distribution:")
print(y_test.value_counts(normalize=True))

```

```

Training class distribution:
Outcome
0    0.626374
1    0.373626
Name: proportion, dtype: float64

Testing class distribution:
Outcome
0    0.631579
1    0.368421
Name: proportion, dtype: float64

```

```

# standardizing features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

```

```

print("Mean of first 5 standardized features:")
print(X_train_scaled[:, :5].mean(axis=0))

print("\nStandard deviation of first 5 standardized features:")
print(X_train_scaled[:, :5].std(axis=0))

Mean of first 5 standardized features:
[-1.73731603e-16  3.90408097e-16  4.70441756e-16 -1.17122429e-16
  7.24207019e-16]

Standard deviation of first 5 standardized features:
[1.  1.  1.  1.  1.]

```

```

# model training

logistic_model = LogisticRegression(
    random_state=42,
    max_iter=1000
)

logistic_model.fit(
    X_train_scaled,
    y_train
)

print("model trained.")

```

```
model trained.
```

```

# predictions

y_pred = logistic_model.predict(X_test_scaled)

print("Predictions:")
print(y_pred)

```

```

Predictions:
[0  1  0  1  1  0  1  0  0  0  1  0  1  0  0  0  0  0  0  0  0  1  1  1  0  0  1  1  1  1  0  1  1  0  0  0
  0  1  0  0  0  1  0  0  1  0  1  0  0  1  1  0  0  0  1  0  0  0  1  0  0  1  0  0  0  1  1  0  1  0  1  1  0
  0  0  0  1  0  1  1  1  0  0  0  0  1  0  1  0  1  0  0  0  0  0  1  0  0  0  0  0  0  0  0  1  0  1  0  1  0  0
  0  0  1]

```

```

# accuracy

accuracy = accuracy_score(y_test, y_pred)

```

```
print("Accuracy:", accuracy)
print("Accuracy (%):", accuracy * 100)
```

```
Accuracy: 0.9649122807017544
Accuracy (%): 96.49122807017544
```

```
# precision

precision = precision_score(
    y_test,
    y_pred
)

print("Precision:", precision)
```

```
Precision: 0.975
```

```
# recall

recall = recall_score(
    y_test,
    y_pred
)

print("Recall:", recall)
```

```
Recall: 0.9285714285714286
```

```
# f1 score

f1 = f1_score(
    y_test,
    y_pred
)

print("F1 Score:", f1)
```

```
F1 Score: 0.9512195121951219
```

```
# evaluation for both classes

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Benign", "Malignant"]
    )
)
```

	precision	recall	f1-score	support
Benign	0.96	0.99	0.97	72
Malignant	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

```
# confusion matrix
```

```
cm = confusion_matrix(
    y_test,
    y_pred
)

print(cm)
```

```
[[71  1]
 [ 3 39]]
```

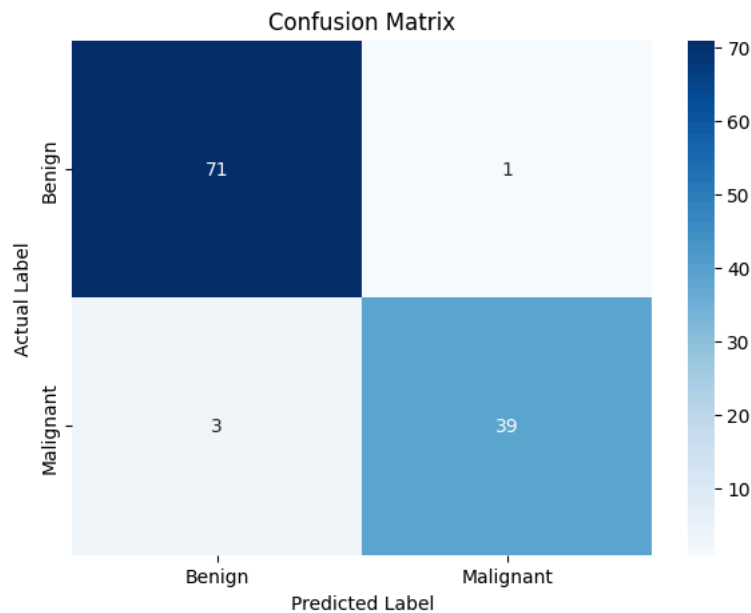
```
plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)
```

```
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix")

plt.show()
```



```
# ROC curve
```

```
y_prob = logistic_model.predict_proba(
    X_test_scaled
)[: , 1]
```

```
print(y_prob[:10])
```

```
[3.64568116e-04 9.99999989e-01 4.25679471e-02 5.76187322e-01
 5.09155489e-01 6.16001620e-04 7.61534086e-01 1.01217174e-03
 5.68725516e-04 1.07675795e-02]
```

```
# caluculating ROC and AUC
```

```
fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob
)
```

```
auc_score = roc_auc_score(
    y_test,
    y_prob
)
```

```
print("ROC-AUC Score:", auc_score)
```

```
ROC-AUC Score: 0.996031746031746
```

```
# plotting ROC curve
```

```
plt.figure(figsize=(8, 6))
```

```
plt.plot(
    fpr,
    tpr,
    label=f"Logistic Regression (AUC = {auc_score:.3f})"
)
```

```
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.title("ROC Curve")  
plt.legend()
```

```
plt.show()
```

